实验原理

1. 预取指令 `prefetch`

x86-64指令集架构包含SSE（Streaming SIMD Extension）扩展指令。SSE扩展指令支持以SIMD方式实现数据的并行处理，有效提高了CPU的数据处理效率。

SSE提供了用于把数据预取到Cache中的预取指令 `prefetch`，如表2-1所示。

表2-1 SSE扩展的预取指令

指令名称	指令功能	适用对象
`prefetcht0`	预取数据到所有层级的Cache中	短期内多次访问的数据
`prefetcht1`	预取数据到L2 Cache及更高层级的Cache中	短期内多次访问的数据
`prefetcht2`	预取数据到L2 Cache及更高层级的Cache中	短期内多次访问的数据
`prefetchnta`	预取数据到某个层级的Cache（尽可能不影响Cache现有数据）	短期内不需多次访问的数据

> **`prefetcht1` 和 `prefetcht2` 的区别**
>
> 以Intel处理器为例，厂商的软件开发手册对 `prefetcht1` 和 `prefetcht2` 的描述都是“fetch data into level 2 cache and higher”。实际 `prefetcht1` 会比 `prefetcht2` 更加倾向于把数据预取到更高层级的Cache，可以简单理解为，`prefetcht1` 尽量把数据预取到L2 Cache，而 `prefetcht2` 则更多时候会把数据预取到L3 Cache。

预取指令的使用方法是 `prefetch<t0/t1/t2/nta> (<byte_addr>)`。其中，`byte_addr` 是 **字节地址**；预取的数据单元大小取决于具体的处理器实现，一般至少为32字节（通常是一个Cache缓存行的大小）。

预取指令的执行是非阻塞的，即 `prefetch` 指令之后的指令不需要等待数据预取操作完成之后再执行。

2. 分块矩阵乘法与缓存优化

目前主流线性代数库大多采用如图2-1所示的Goto算法及其变种算法计算矩阵乘法。Goto算法的核心是根据分块矩阵乘法算法的计算流程，将参与计算的数据块存入不同的缓存以期将数据的访问时间隐藏在分块计算中。

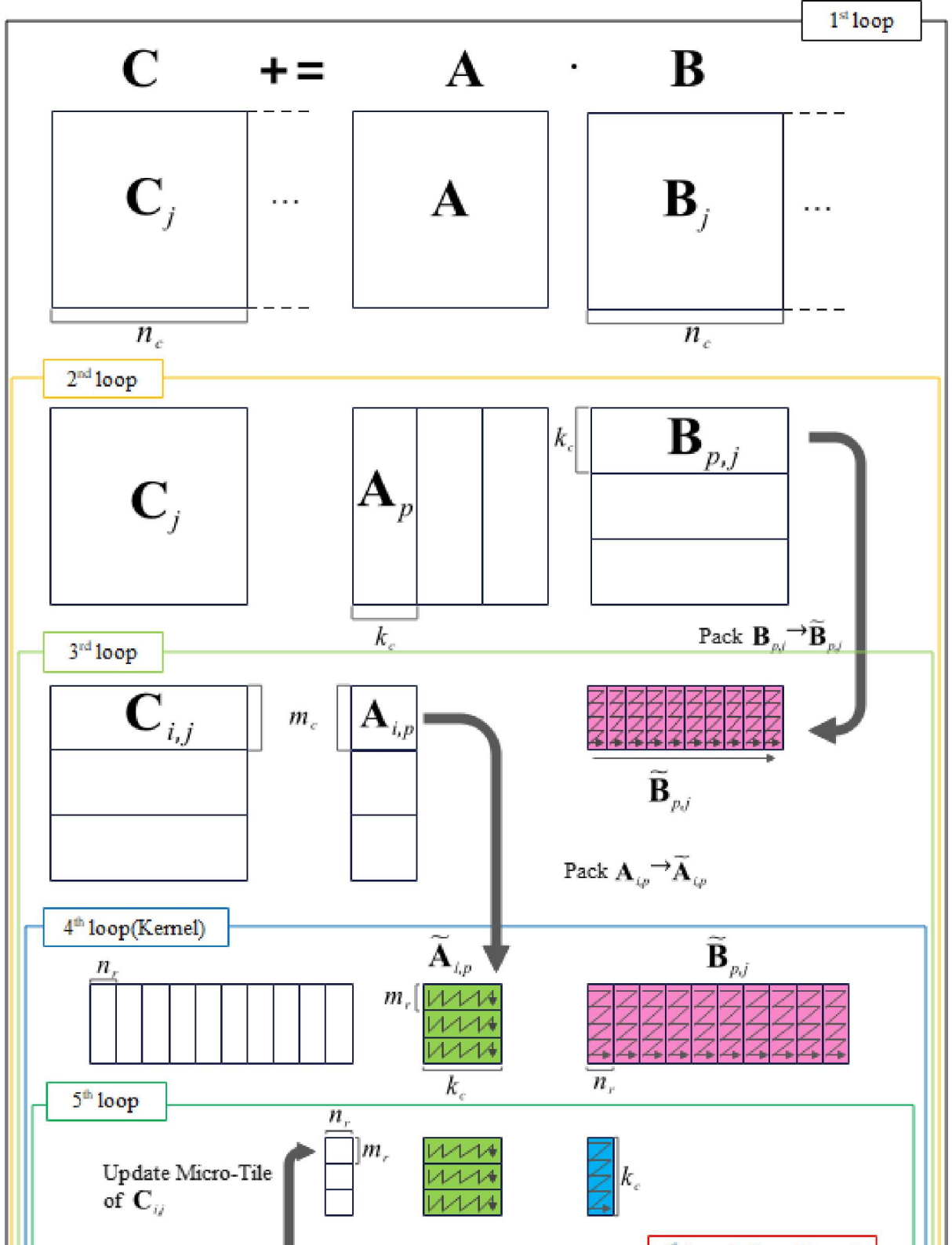
经典的Goto算法采用如图2-1所示的六重循环，第4层循环到第6层循环通常被称为Kernel（内核），为了保证计算的效率，一般采用手写汇编的方式进行实现。

图2-1中各参数的一般设置规律如下：

- 第6层循环中的 m_r, n_r 与处理器核心中可使用的寄存器数量强相关。通常会将绝大多数的数据寄存器分给和矩阵 C 的分块（$m_r \times n_r$ 大小），以保证有足够的时间从L2缓存按行加载计算所需的矩阵 A 的数据，从L1D缓存加载矩阵 B 的数据。
- 第5层循环中 k_c 的设定一般需考虑 $k_c \times n_r$ 大小的矩阵 B 的数据能够占据L1D缓存的多数way，余下的way供矩阵 C 和矩阵 A 使用。此时矩阵 B 的数据会因为重复使用而停留在L1D缓存中。若 k_c 设置得过小，会导致第2层循环中矩阵 C 的部分和的累加次数增多，由于矩阵 C 保存在内存中，过多的累加会严重的拖慢计算速度。
- 第3层循环中 m_c 的认定一般需考虑 $m_c \times k_c$ 大小的矩阵 A 的数据能够占据L2缓存的多数way，计算时数据直接从L2缓存流向寄存器。
- 第1重循环中 n_c 的选择需保证第3层循环中矩阵 B 的数据能够占据L3绝大部分way。

因为对矩阵进行分块后会带来分块元素访问不连续的问题，所以，通常情况下会在第3重循环和第4层循环对分块数据进行Packing以保证Kernel访问元素的连续性和空间局部性。需注意的是Packing本身有代价，并不总是能带来正向的收益，灵活的packing策略有助于提高矩阵乘法的计算性能。

主流处理器的计算单元和访存单元通常为独立部件，这意味着计算的同时可以进行访存相关的操作。如果能够根据算法的计算流程和各级存储的访问代价(比如延迟，带宽)将矩阵乘法下一次所需的数据提前准备到对应位置，能大幅减少数据的读取时间获得更高的计算性能。需要注意提前准备数据不能引发剧烈的way冲突，也不能带来更严重的缓存驱逐。


1st loop
C += A · B
C_j
A
B_j
n_c
2nd loop
A_p
k_c
$B_{p,j}$
Pack $B_{p,j} \rightarrow \widetilde{B}_{p,j}$
3rd loop
$C_{i,j}$
m_c
$A_{i,p}$
$\widetilde{B}_{p,j}$
Pack $A_{ip} \rightarrow \widetilde{A}_{ip}$
4th loop(Kernel)
n_r
$\widetilde{A}_{i,p}$
m_r
5th loop
Update Micro-Tile of C_{ij}

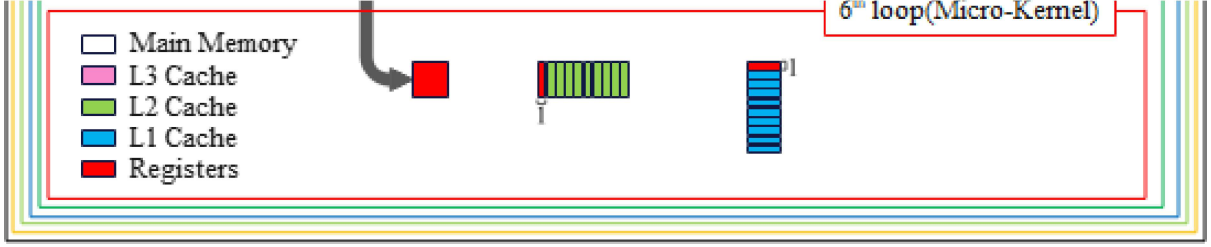


图2-1 Goto算法的一般流程

若需更系统的了解高性能矩阵乘法的设计和优化方法，请查阅以下资料：

- 《Anatomy of High-Performance Matrix Multiplication》
- 《Theory and Practice of Classical Matrix-Matrix Multiplication for Hierarchical Memory Architectures》
- 《Analytical modeling is enough for high-performance BLIS》